Taller — Modelos de probabilidad de default

Andrés Londoño

Contents

1. Regresión logística `log_model_ir` (único predictor: `ir_cat`) . . . 2
2. Parámetros estimados . . . 2
3. Categoría de referencia de `ir_cat` . . . 2
4. Regresión logística múltiple `log_model_multi` . . . 3
5. Variables significativas, odds y efectos marginales . . . 4
6. Modelo `log_model_age_ir` . . . 5
7. Predicciones sobre `test_set`: `predictions_all_small` . . . 6
8. Rango de las predicciones: ¿qué tan bien discrimina? . . . 6
9. Modelo completo `log_model_full` (sin `int_rate` ni `emp_length`) . . . 6
10. Predicciones del modelo completo . . . 7
11. Rango de las probabilidades: modelo completo vs. `log_model_age_ir` . . . 7
12. Modelos `probit_model_full` y `rf_model_full` . . . 8
13. Pronósticos y vector de clasificación con corte 0,14 . . . 8
14. Matrices de confusión . . . 9
15. Precisión, sensibilidad y especificidad . . . 10
16. Curvas ROC . . . 10
17. AUC y mejor modelo . . . 11
18. Corte óptimo del mejor modelo (máxima distancia TPR − FPR, índice de Youden) . . . 12

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from pandas.api.types import CategoricalDtype
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn import metrics

link = "https://raw.githubusercontent.com/salas317/Data/main/GRF/loan_data.csv"
loan_data = pd.read_csv(link, index_col=0)

loan_data["home_ownership"] = loan_data["home_ownership"].astype("category")
loan_data["grade"] = loan_data["grade"].astype(CategoricalDtype(categories=list("GFEDCBA"), ordered=Tru
```

```
# "Missing" va primero: es la categoría de referencia de ir_cat en los modelos
loan_data["ir_cat"] = loan_data["ir_cat"].astype(
    CategoricalDtype(categories=["Missing", "0-8", "8-11", "11-13.5", "13.5+"], ordered=True))

train_set, test_set = train_test_split(loan_data, test_size=1/3, random_state=567)
print(train_set.shape, test_set.shape)
```

```
## (19394, 10) (9697, 10)
```

1. Regresión logística `log_model_ir` (único predictor: `ir_cat`)

```
log_model_ir = smf.logit("loan_status ~ ir_cat", data=train_set).fit()
```

```
## Optimization terminated successfully.
##          Current function value: 0.338767
##          Iterations 7
```

2. Parámetros estimados

```
print(log_model_ir.summary())
```

```
##                            Logit Regression Results
## ==============================================================================
## Dep. Variable:            loan_status   No. Observations:                19394
## Model:                          Logit   Df Residuals:                    19389
## Method:                           MLE   Df Model:                            4
## Date:                dom, 11 oct. 2026   Pseudo R-squ.:                 0.02499
## Time:                        14:40:46   Log-Likelihood:                -6570.0
## converged:                       True   LL-Null:                       -6738.4
## Covariance Type:            nonrobust   LLR p-value:                 1.292e-71
## ===================================================================================
##                       coef    std err          z      P>|z|      [0.025      0.975]
## -----------------------------------------------------------------------------------
## Intercept          -2.1512      0.077    -28.064      0.000      -2.301      -2.001
## ir_cat[T.0-8]      -0.6887      0.099     -6.927      0.000      -0.884      -0.494
## ir_cat[T.8-11]     -0.0677      0.093     -0.732      0.464      -0.249       0.114
## ir_cat[T.11-13.5]   0.2251      0.089      2.541      0.011       0.051       0.399
## ir_cat[T.13.5+]     0.5846      0.087      6.700      0.000       0.414       0.756
## ===================================================================================
```

3. Categoría de referencia de `ir_cat`

```
print(loan_data["ir_cat"].value_counts())
```

```
## ir_cat
## 0-8        7130
## 11-13.5    6953
## 8-11       6230
## 13.5+      6002
## Missing    2776
## Name: count, dtype: int64
```

```
print("Categoría de referencia del modelo:", loan_data["ir_cat"].cat.categories[0])
```

```
## Categoría de referencia del modelo: Missing
```

Respuesta: la categoría de referencia es **Missing**: es la primera categoría de `ir_cat` y por eso `statsmodels` la deja fuera del modelo, mientras las demás se interpretan *respecto a ella*. Según `value_counts`, la categoría más frecuente es `0-8` (7.130 préstamos) y la menos frecuente es `Missing` (2.776), así que la referencia no es la más frecuente sino la que se fijó con el orden de las categorías. En el modelo de un solo predictor: `0-8` tiene un coeficiente de $-0{,}689$ (odds de default 0,50 veces los de Missing), `8-11` no es significativo ($p = 0{,}464$), `11-13.5` sube el log-odds en 0,225 y `13.5+` en 0,585 (odds 1,79 veces). A mayor tasa de interés, mayor probabilidad de default.

4. Regresión logística múltiple `log_model_multi`

```
log_model_multi = smf.logit("loan_status ~ age + ir_cat + grade + loan_amnt + annual_inc", data=train_s
```

```
## Optimization terminated successfully.
##          Current function value: 0.334827
##          Iterations 7
```

```
print(log_model_multi.summary())
```

```
##                            Logit Regression Results
## ==============================================================================
## Dep. Variable:            loan_status   No. Observations:                19394
## Model:                          Logit   Df Residuals:                    19380
## Method:                           MLE   Df Model:                           13
## Date:                dom, 11 oct. 2026   Pseudo R-squ.:                0.03633
## Time:                        14:40:47   Log-Likelihood:                -6493.6
## converged:                       True   LL-Null:                       -6738.4
## Covariance Type:            nonrobust   LLR p-value:                 2.420e-96
## ===========================================================================================
##                               coef    std err          z      P>|z|      [0.025      0.975]
## -------------------------------------------------------------------------------------------
## Intercept                  -0.1811      0.392     -0.463      0.644      -0.949       0.586
## ir_cat[T.0-8]              -0.4171      0.127     -3.290      0.001      -0.666      -0.169
## ir_cat[T.8-11]              0.0734      0.099      0.740      0.459      -0.121       0.268
## ir_cat[T.11-13.5]           0.1940      0.094      2.064      0.039       0.010       0.378
## ir_cat[T.13.5+]             0.2390      0.104      2.303      0.021       0.036       0.442
## grade[T.F]                 -0.2815      0.406     -0.693      0.489      -1.078       0.515
## grade[T.E]                 -0.7826      0.372     -2.104      0.035      -1.512      -0.054
## grade[T.D]                 -1.0266      0.362     -2.837      0.005      -1.736      -0.317
```

```
## grade[T.C]           -1.2171      0.363     -3.350      0.001      -1.929      -0.505
## grade[T.B]           -1.4877      0.369     -4.028      0.000      -2.212      -0.764
## grade[T.A]           -1.6604      0.380     -4.375      0.000      -2.404      -0.917
## age                  -0.0075      0.004     -1.909      0.056      -0.015       0.000
## loan_amnt         -1.234e-06    4.2e-06     -0.294      0.769   -9.46e-06    6.99e-06
## annual_inc        -5.927e-06   7.59e-07     -7.812      0.000   -7.41e-06   -4.44e-06
## ==============================================================================
```

5. Variables significativas, odds y efectos marginales

```
pvals = log_model_multi.pvalues
signif = pvals[pvals < 0.05].drop("Intercept", errors="ignore")
print("Significativos al 5%:\n", signif.round(4), "\n")
```

```
## Significativos al 5%:
##  ir_cat[T.0-8]        0.0010
## ir_cat[T.11-13.5]    0.0390
## ir_cat[T.13.5+]      0.0213
## grade[T.E]           0.0354
## grade[T.D]           0.0046
## grade[T.C]           0.0008
## grade[T.B]           0.0001
## grade[T.A]           0.0000
## annual_inc           0.0000
## dtype: float64
```

```
odds = pd.DataFrame({"coef": log_model_multi.params, "odds = exp(coef)": np.exp(log_model_multi.params)})
print(odds.round(4))
```

```
##                      coef  odds = exp(coef)
## ir_cat[T.0-8]     -0.4171            0.6589
## ir_cat[T.11-13.5]  0.1940            1.2141
## ir_cat[T.13.5+]    0.2390            1.2700
## grade[T.E]        -0.7826            0.4572
## grade[T.D]        -1.0266            0.3582
## grade[T.C]        -1.2171            0.2961
## grade[T.B]        -1.4877            0.2259
## grade[T.A]        -1.6604            0.1901
## annual_inc        -0.0000            1.0000
```

```
print()
```

```
me = log_model_multi.get_margeff().summary_frame()
print(me.loc[me.index.isin(signif.index)].round(5))
```

```
##                      dy/dx  Std. Err.  ...  Conf. Int. Low  Cont. Int. Hi.
## ir_cat[T.0-8]     -0.03993    0.01215  ...        -0.06374        -0.01612
## ir_cat[T.11-13.5]  0.01857    0.00900  ...         0.00094         0.03620
## ir_cat[T.13.5+]    0.02288    0.00993  ...         0.00341         0.04235
## grade[T.E]        -0.07491    0.03560  ...        -0.14469        -0.00514
```

```
## grade[T.D]        -0.09827    0.03463  ...        -0.16615        -0.03039
## grade[T.C]        -0.11651    0.03477  ...        -0.18466        -0.04836
## grade[T.B]        -0.14241    0.03535  ...        -0.21169        -0.07313
## grade[T.A]        -0.15894    0.03632  ...        -0.23013        -0.08775
## annual_inc        -0.00000    0.00000  ...        -0.00000        -0.00000
##
## [9 rows x 6 columns]
```

```
print()
```

```
b = log_model_multi.params["annual_inc"]
print(f"annual_inc, por cada 10.000 adicionales -> odds: {np.exp(b*1e4):.4f} | efecto marginal: {me.loc
```

```
## annual_inc, por cada 10.000 adicionales -> odds: 0.9425 | efecto marginal: -0.0057
```

Significativas al 5%: `ir_cat` (0-8, 11-13.5 y 13.5+), `grade` (E, D, C, B y A) y `annual_inc`. **No significativas:** `age` (p = 0,056, queda en el límite), `loan_amnt` (p = 0,769), `ir_cat` 8-11 y `grade` F.

- **Tasa de interés** (referencia Missing): `0-8` multiplica los odds por 0,659 (−34 %) y baja la probabilidad de default 4,0 pp; `11-13.5` los multiplica por 1,214 (+1,9 pp) y `13.5+` por 1,270 (+2,3 pp).
- **Calificación** (referencia G, la peor): cuanto mejor la calificación, menor el riesgo. Los odds se multiplican por 0,457 (E), 0,358 (D), 0,296 (C), 0,226 (B) y 0,190 (A). En probabilidad, la baja es de 7,5 pp (E) hasta 15,9 pp (A) respecto de la categoría G.
- **Ingreso anual:** cada 10.000 adicionales multiplican los odds por 0,9425 (−5,8 %) y reducen la probabilidad de default 0,57 pp. A mayor ingreso, menos riesgo.

Todo *ceteris paribus*.

6. Modelo `log_model_age_ir`

```
log_model_age_ir = smf.logit("loan_status ~ age + ir_cat", data=train_set).fit()
```

```
## Optimization terminated successfully.
##          Current function value: 0.338539
##          Iterations 7
```

```
print(log_model_age_ir.summary())
```

```
##                            Logit Regression Results
## ==============================================================================
## Dep. Variable:            loan_status   No. Observations:                19394
## Model:                          Logit   Df Residuals:                    19388
## Method:                           MLE   Df Model:                            5
## Date:                dom, 11 oct. 2026   Pseudo R-squ.:                 0.02564
## Time:                        14:40:47   Log-Likelihood:                -6565.6
## converged:                       True   LL-Null:                       -6738.4
## Covariance Type:            nonrobust   LLR p-value:                 1.597e-72
## ==============================================================================
##                          coef    std err          z      P>|z|      [0.025      0.975]
```

```
## ------------------------------------------------------------------------------
## Intercept           -1.8338      0.132    -13.841      0.000      -2.093      -1.574
## ir_cat[T.0-8]       -0.6909      0.099     -6.948      0.000      -0.886      -0.496
## ir_cat[T.8-11]      -0.0689      0.093     -0.744      0.457      -0.250       0.113
## ir_cat[T.11-13.5]    0.2232      0.089      2.519      0.012       0.050       0.397
## ir_cat[T.13.5+]      0.5849      0.087      6.702      0.000       0.414       0.756
## age                 -0.0115      0.004     -2.918      0.004      -0.019      -0.004
## ==============================================================================
```

7. Predicciones sobre `test_set`: `predictions_all_small`

```
predictions_all_small = log_model_age_ir.predict(exog=test_set)
predictions_all_small.head()
```

```
## 25348    0.131691
## 28805    0.055482
## 24143    0.134339
## 7210     0.156426
## 16242    0.053705
## dtype: float64
```

8. Rango de las predicciones: ¿qué tan bien discrimina?

```
print("Mínimo:", predictions_all_small.min().round(4), " Máximo:", predictions_all_small.max().round(4))
```

```
## Mínimo: 0.0362  Máximo: 0.1857
```

```
print("Rango:", (predictions_all_small.max() - predictions_all_small.min()).round(4))
```

```
## Rango: 0.1495
```

Respuesta: las probabilidades predichas van de 3,6 % a 18,6 % (rango de 0,15). El modelo nunca asigna una probabilidad alta de default, aunque algunos préstamos sí caen en default, y sus predicciones están encerradas en una banda estrecha alrededor de la tasa base (11 %). Eso indica una **capacidad de discriminación limitada**: separa algo a los clientes (el pseudo-R^2 es solo 0,026), pero no identifica con claridad quién va a caer en default.

9. Modelo completo `log_model_full` (sin `int_rate` ni `emp_length`)

```
f_full = "loan_status ~ loan_amnt + grade + home_ownership + annual_inc + age + ir_cat"
log_model_full = smf.logit(f_full, data=train_set).fit()
```

```
## Optimization terminated successfully.
##          Current function value: 0.334780
##          Iterations 7
```

```
print(log_model_full.summary())
```

```
##                            Logit Regression Results
## ==============================================================================
## Dep. Variable:            loan_status   No. Observations:                19394
## Model:                          Logit   Df Residuals:                    19377
## Method:                           MLE   Df Model:                           16
## Date:                dom, 11 oct. 2026   Pseudo R-squ.:                0.03646
## Time:                        14:40:48   Log-Likelihood:                -6492.7
## converged:                       True   LL-Null:                       -6738.4
## Covariance Type:            nonrobust   LLR p-value:                2.228e-94
## ================================================================================================
##                                  coef    std err          z      P>|z|      [0.025      0.975]
## ------------------------------------------------------------------------------------------------
## Intercept                     -0.1404      0.394     -0.356      0.722      -0.913       0.633
## grade[T.F]                    -0.2817      0.407     -0.693      0.488      -1.079       0.515
## grade[T.E]                    -0.7821      0.372     -2.102      0.036      -1.511      -0.053
## grade[T.D]                    -1.0264      0.362     -2.835      0.005      -1.736      -0.317
## grade[T.C]                    -1.2174      0.363     -3.349      0.001      -1.930      -0.505
## grade[T.B]                    -1.4899      0.369     -4.033      0.000      -2.214      -0.766
## grade[T.A]                    -1.6677      0.380     -4.393      0.000      -2.412      -0.924
## home_ownership[T.OTHER]        0.2705      0.352      0.767      0.443      -0.420       0.961
## home_ownership[T.OWN]         -0.0075      0.091     -0.082      0.935      -0.186       0.171
## home_ownership[T.RENT]        -0.0528      0.052     -1.008      0.313      -0.155       0.050
## ir_cat[T.0-8]                 -0.4169      0.127     -3.286      0.001      -0.666      -0.168
## ir_cat[T.8-11]                 0.0753      0.099      0.759      0.448      -0.119       0.270
## ir_cat[T.11-13.5]              0.1956      0.094      2.081      0.037       0.011       0.380
## ir_cat[T.13.5+]                0.2415      0.104      2.326      0.020       0.038       0.445
## loan_amnt                  -1.381e-06    4.2e-06     -0.329      0.742   -9.61e-06    6.85e-06
## annual_inc                 -6.118e-06   7.91e-07     -7.733      0.000   -7.67e-06   -4.57e-06
## age                           -0.0074      0.004     -1.899      0.058      -0.015       0.000
## ================================================================================================
```

10. Predicciones del modelo completo

```
predictions_all_full = log_model_full.predict(exog=test_set)
predictions_all_full.head()
```

```
## 25348    0.148418
## 28805    0.057194
## 24143    0.132494
## 7210     0.175834
## 16242    0.044036
## dtype: float64
```

11. Rango de las probabilidades: modelo completo vs. `log_model_age_ir`

```python
rangos = pd.DataFrame({
    "log_model_age_ir": [predictions_all_small.min(), predictions_all_small.max()],
    "log_model_full": [predictions_all_full.min(), predictions_all_full.max()]}, index=["mínimo", "máxi
rangos.loc["rango"] = rangos.loc["máximo"] - rangos.loc["mínimo"]
rangos.round(4)
```

```
##         log_model_age_ir  log_model_full
## mínimo            0.0362          0.0000
## máximo            0.1857          0.4382
## rango             0.1495          0.4382
```

Respuesta: con el modelo completo el rango se ensancha: de 0,0 % a 43,8 % (rango de 0,44), frente a 3,6 %–18,6 % del modelo `log_model_age_ir`. Al incorporar la calificación y el ingreso anual, el modelo se atreve a asignar probabilidades mucho más bajas y mucho más altas, es decir, distingue mejor entre buenos y malos pagadores. Aun así la mejora es moderada (pseudo-R^2 de 0,036).

12. Modelos `probit_model_full` y `rf_model_full`

```python
probit_model_full = smf.probit(f_full, data=train_set).fit()
```

```
## Optimization terminated successfully.
##          Current function value: 0.334918
##          Iterations 6
```

```python
cols = ["loan_amnt", "grade", "home_ownership", "annual_inc", "age", "ir_cat"]
X_train = pd.get_dummies(train_set[cols], columns=["grade", "home_ownership", "ir_cat"], dtype=int)
X_test = pd.get_dummies(test_set[cols], columns=["grade", "home_ownership", "ir_cat"], dtype=int)
y_train, y_test = train_set["loan_status"], test_set["loan_status"]

rf_model_full = RandomForestClassifier(n_estimators=500, random_state=567)
rf_model_full.fit(X_train, y_train)
```

```
## RandomForestClassifier(n_estimators=500, random_state=567)
```

13. Pronósticos y vector de clasificación con corte 0,14

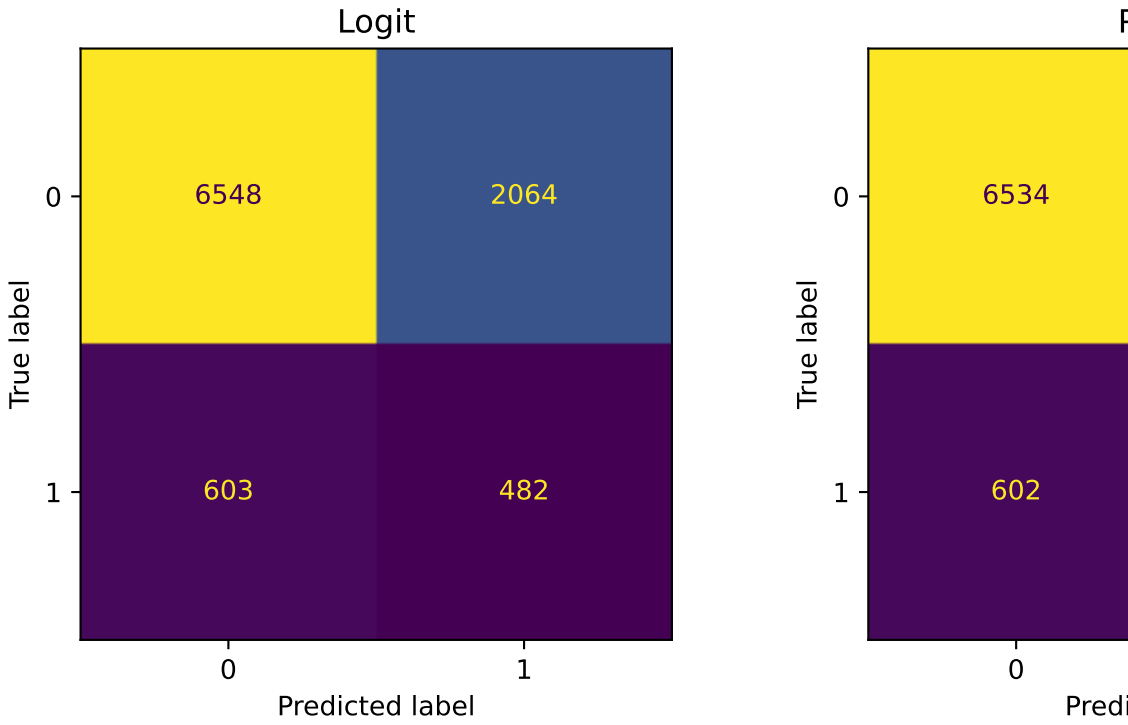
```python
probs_logit = log_model_full.predict(exog=test_set)
probs_probit = probit_model_full.predict(exog=test_set)
probs_rf = rf_model_full.predict_proba(X_test)[:, 1]

corte = 0.14
preds_logit = (probs_logit >= corte).astype(int)
preds_probit = (probs_probit >= corte).astype(int)
preds_rf = (probs_rf >= corte).astype(int)
print("Predichos como default:", preds_logit.sum(), preds_probit.sum(), preds_rf.sum(), "de", len(y_tes
```

```
## Predichos como default: 2546 2561 2959 de 9697
```

14. Matrices de confusión

```
preds = {"Logit": preds_logit, "Probit": preds_probit, "Random forest": preds_rf}
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, (nombre, p) in zip(axes, preds.items()):
    metrics.ConfusionMatrixDisplay(metrics.confusion_matrix(y_test, p)).plot(ax=ax, colorbar=False)
    ax.set_title(nombre)
plt.show()
```




```
for nombre, p in preds.items():
    print(nombre); print(pd.crosstab(y_test, p, rownames=["Real"], colnames=["Predicho"])); print()
```

```
## Logit
## Predicho     0     1
## Real
## 0         6548  2064
## 1          603   482
##
## Probit
## Predicho     0     1
## Real
## 0         6534  2078
## 1          602   483
##
## Random forest
## Predicho     0     1
```

```
## Real
## 0         6079  2533
## 1          659   426
```

15. Precisión, sensibilidad y especificidad

```
res = pd.DataFrame({
    nombre: {"Precisión": metrics.accuracy_score(y_test, p),
             "Sensibilidad": metrics.recall_score(y_test, p),
             "Especificidad": metrics.recall_score(y_test, p, pos_label=0)}
    for nombre, p in preds.items()})
(res * 100).round(2)
```

```
##                Logit  Probit  Random forest
## Precisión      72.50   72.36          67.08
## Sensibilidad   44.42   44.52          39.26
## Especificidad  76.03   75.87          70.59
```

Respuesta: con corte de 0,14, logit y probit son prácticamente idénticos (precisión 72,5 %, sensibilidad 44,4 %, especificidad 76,0 %). El bosque aleatorio es peor en las tres medidas (67,1 %, 39,3 % y 70,6 %). Como solo 11 % de los préstamos caen en default, la precisión global engaña un poco: lo que más importa a un prestamista es la sensibilidad, y con este corte se deja pasar a más de la mitad de los que luego incumplen.

16. Curvas ROC

```
probs = {"Logit": probs_logit, "Probit": probs_probit, "Random forest": probs_rf}
plt.figure(figsize=(8, 6))
for nombre, pr in probs.items():
    fpr, tpr, _ = metrics.roc_curve(y_test, pr)
    plt.plot(fpr, tpr, label=nombre)
```

```
## [<matplotlib.lines.Line2D object at 0x00000222441FF230>]
## [<matplotlib.lines.Line2D object at 0x00000222441FF380>]
## [<matplotlib.lines.Line2D object at 0x00000222441FF4D0>]
```

```
plt.plot([0, 1], [0, 1], "k--", label="Azar")
```

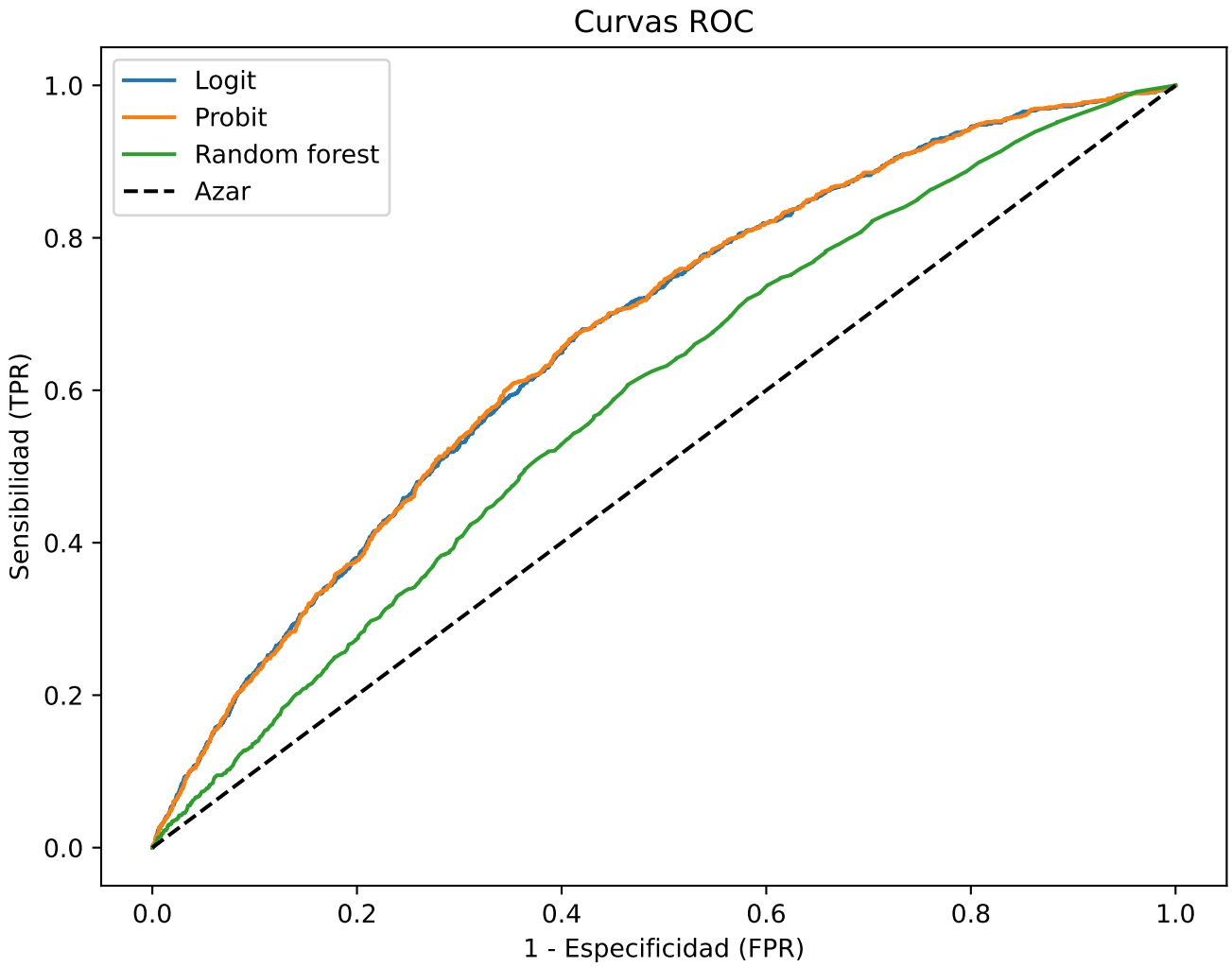
```
## [<matplotlib.lines.Line2D object at 0x00000222441FF620>]
```

```
plt.xlabel("1 - Especificidad (FPR)"); plt.ylabel("Sensibilidad (TPR)"); plt.title("Curvas ROC")
```

```
## Text(0.5, 0, '1 - Especificidad (FPR)')
## Text(0, 0.5, 'Sensibilidad (TPR)')
## Text(0.5, 1.0, 'Curvas ROC')
```

```
plt.legend(); plt.show()
```

```
## <matplotlib.legend.Legend object at 0x00000222441FF0E0>
```



17. AUC y mejor modelo

```
auc = {n: metrics.roc_auc_score(y_test, pr) for n, pr in probs.items()}
print(pd.Series(auc).round(4))
```

```
## Logit            0.6680
## Probit           0.6686
## Random forest    0.5899
## dtype: float64
```

```
mejor = max(auc, key=auc.get)
print("Mejor modelo por AUC:", mejor)
```

```
## Mejor modelo por AUC: Probit
```

Respuesta: el AUC del probit (0,6686) es el más alto, prácticamente empatado con el logit (0,6680). El bosque aleatorio queda en 0,5899, bastante más cerca del azar (0,5). Para estos datos el **probit** es el modelo más adecuado, aunque la diferencia con el logit es insignificante. El bosque, con 500 árboles sin ajustar, parece sobreajustar los datos de entrenamiento y no generaliza.

18. Corte óptimo del mejor modelo (máxima distancia TPR − FPR, índice de Youden)

```
fpr, tpr, thr = metrics.roc_curve(y_test, probs[mejor])
k = np.argmax(tpr - fpr)
print(f"Corte óptimo ({mejor}): {thr[k]:.4f} | TPR = {tpr[k]:.4f} | FPR = {fpr[k]:.4f} | TPR-FPR = {tpr
```

```
## Corte óptimo (Probit): 0.1132 | TPR = 0.6747 | FPR = 0.4145 | TPR-FPR = 0.2601
```

Respuesta: para el probit, el corte que maximiza la distancia TPR − FPR (índice de Youden) es **0,1132**, con TPR = 67,5 %, FPR = 41,4 % y distancia de 0,260. Es menor que el 0,14 usado antes: clasifica más préstamos como default, así que la sensibilidad sube de 44,5 % a 67,5 % a cambio de más falsos positivos (rechazar a buenos clientes). La decisión final dependería del costo relativo de un default frente al de rechazar a un buen cliente.